

Escenarios de casos de uso - GameVault

Basado en el código real del repositorio (backend Django + DRF). Cada escenario documenta un flujo de ÉXITO o de FALLO según el comportamiento efectivo de las vistas y serializers.

Nombre del caso de uso Registro de una cuenta nueva	
ID # #1.1	
Actores Visitante	
Precondiciones El visitante no tiene una cuenta activa en el sistema. No existe un registro pendiente previo asociado a ese email.	
Suposiciones El servicio de email está operativo. El visitante completa el formulario con datos válidos.	
Evento que desencadena el escenario El visitante envía el formulario de registro (POST /api/users/register/) con username, email, password y password_confirm.	
Tipo de Evento Externo, iniciado por el actor (Visitante) desde el frontend.	
Garantía de éxito (ÉXITO) Se crea un registro en PendingRegistration con la contraseña ya hasheada (make_password) y se genera un token único (secrets.token_urlsafe). El sistema envía un email de confirmación. El usuario final (User) todavía NO existe hasta que se confirme el email.	
Paso a paso	
Paso 1	El visitante completa el formulario con username, email, password y password_confirm.
Paso 2	El sistema valida el serializer (RegisterSerializer): password y password_confirm deben coincidir, y la contraseña debe pasar las validaciones de Django (validate_password).
Paso 3	El sistema verifica que no exista ya un User con ese email.
Paso 4	El sistema elimina cualquier PendingRegistration previo con el mismo email, para evitar registros pendientes duplicados.
Paso 5	El sistema genera un token seguro con secrets.token_urlsafe(32).
Paso 6	El sistema hace hash de la contraseña con make_password antes de guardarla.
Paso 7	El sistema crea el registro PendingRegistration (username, email, password hasheada, token).
Paso 8	El sistema envía el email de verificación con el link de confirmación.
Paso 9	El sistema responde 201 con el mensaje "Registro iniciado. Revisa tu correo para confirmar tu cuenta." y el flag email_sent.
Post Condiciones Existe un PendingRegistration en estado de espera. No existe todavía un User real hasta que se confirme el email (ver #2.1).	

Cuestiones pendientes

El endpoint devuelve verification_url directamente en la respuesta (util para testing, pero en produccion expone el link de confirmacion en el propio response).

Nombre del caso de uso

Registro con un email ya utilizado

ID # #1.2

Actores

Visitante

Precondiciones

Ya existe un User activo registrado con ese email.

Suposiciones

El visitante no sabe que el email ya esta en uso, o lo olvido.

Evento que desencadena el escenario

El visitante envia el formulario de registro con un email que ya pertenece a un User existente.

Tipo de Evento Externo, iniciado por el actor.

Garantia de exito (FALLO)

No se crea ningun PendingRegistration ni User nuevo. El sistema responde 400 con el mensaje "El email ya esta en uso." y no se envia ningun email.

Paso a paso**Paso 1**

El visitante completa el formulario con un email ya registrado.

Paso 2

El serializer valida el formato de los datos correctamente (formato de email OK, contrasenas coinciden).

Paso 3

El sistema chequea `User.objects.filter(email=...).exists()` y encuentra una coincidencia.

Paso 4

El sistema corta el flujo antes de tocar PendingRegistration.

Paso 5

El sistema responde con status 400, code "validation_error" y detalle en el campo email.

Paso 6

El frontend muestra el error para que el visitante use otro email o recupere su contrasena.

Post Condiciones

No se modifica el estado del sistema. El User existente permanece intacto.

Cuestiones pendientes

El mensaje confirma explicitamente que el email ya existe, lo que podria usarse para enumerar cuentas registradas (a diferencia del flujo de recuperacion de contrasena, ver #4.1, que es discreto).

Nombre del caso de uso Confirmacion de email	
ID # #2.1	
Actores Visitante (con un registro pendiente)	
Precondiciones Existe un PendingRegistration valido, generado en #1.1, con menos de 24 horas de antigüedad.	
Suposiciones El visitante hace clic en el link de confirmacion recibido por email antes de que expire.	
Evento que desencadena el escenario El visitante abre el link de verificacion y el frontend envia POST /api/users/verify-email/ con uid y token.	
Tipo de Evento Externo, iniciado por el actor mediante un link con vencimiento temporal.	
Garantia de exito (EXITO) Se crea el User definitivo, con la contraseña hashheada copiada del PendingRegistration, y este ultimo se elimina. Recien aqui el visitante pasa a ser un usuario real.	
Paso a paso	
Paso 1	El visitante hace clic en el link de verificacion (contiene uid y token).
Paso 2	El sistema busca el PendingRegistration por uid; si no existe, corta el flujo.
Paso 3	El sistema compara el token recibido con el guardado; deben coincidir exactamente.
Paso 4	El sistema valida que no hayan pasado mas de 24 horas desde created_at.
Paso 5	El sistema crea un User nuevo con el username y email del PendingRegistration.
Paso 6	El sistema copia directamente la contraseña ya hashheada, sin volver a hashearla.
Paso 7	El sistema elimina el PendingRegistration.
Paso 8	El sistema responde 201 con "Cuenta confirmada exitosamente. Ya podes iniciar sesion."
Paso 9	El visitante ya puede iniciar sesion como usuario registrado (ver #3.1).
Post Condiciones Existe un User activo en la base de datos. El PendingRegistration correspondiente ya no existe.	
Cuestiones pendientes Ninguna.	

Nombre del caso de uso Confirmacion con token vencido	
ID # #2.2	
Actores Visitante	

Precondiciones Existe un PendingRegistration con mas de 24 horas de antigüedad desde su creación.	
Suposiciones El visitante tardó en revisar su correo o el link quedó abierto en una pestaña vieja.	
Evento que desencadena el escenario El visitante hace clic en un link de verificación vencido.	
Tipo de Evento Externo, con vencimiento temporal (regla de negocio: 24 horas).	
Garantía de éxito (FALLO) No se crea ningún User. El PendingRegistration vencido se elimina automáticamente al detectarse el vencimiento, para no dejar registros pendientes acumulados.	
Paso a paso	
Paso 1	El visitante hace clic en el link de verificación vencido.
Paso 2	El sistema encuentra el PendingRegistration por uid.
Paso 3	El sistema calcula <code>timezone.now() - pending.created_at</code> y detecta que supera las 24 horas.
Paso 4	El sistema elimina el PendingRegistration vencido.
Paso 5	El sistema responde 400 con "El enlace ha expirado. Regístrate nuevamente."
Paso 6	El visitante debe volver a completar el formulario de registro desde cero (ver #1.1).
Post Condiciones El PendingRegistration vencido fue eliminado. El visitante sigue sin cuenta.	
Cuestiones pendientes No existe un endpoint para reenviar el email de verificación sin volver a registrarse por completo; podría ser una mejora a futuro.	

Nombre del caso de uso Inicio de sesion	
ID # #3.1	
Actores Usuario registrado	
Precondiciones El usuario ya confirmo su email (existe como User real).	
Suposiciones El usuario recuerda su username y contrasena correctos.	
Evento que desencadena el escenario El usuario envia sus credenciales a POST /api/users/login/ (TokenObtainPairView de SimpleJWT).	
Tipo de Evento Externo, iniciado por el actor.	
Garantia de exito (EXITO) El sistema entrega un access token (valido 2 horas) y un refresh token (valido 7 dias), permitiendo autenticar los siguientes requests.	
Paso a paso	
Paso 1	El usuario envia username y password.
Paso 2	SimpleJWT valida las credenciales contra la base de datos (compara contra el hash guardado).
Paso 3	El sistema genera un par de tokens JWT: access token y refresh token.
Paso 4	El sistema responde 200 con ambos tokens.
Paso 5	El frontend guarda los tokens y los adjunta en el header Authorization de los siguientes requests.
Post Condiciones El usuario queda autenticado; puede acceder a endpoints protegidos (perfil, mi coleccion, etc.).	
Cuestiones pendientes Ninguna.	

Nombre del caso de uso Inicio de sesion con credenciales invalidas	
ID # #3.2	
Actores Visitante o usuario que se equivoca de contrasena	
Precondiciones El usuario puede existir o no; el resultado es el mismo.	
Suposiciones El actor escribe mal el username o la contrasena.	
Evento que desencadena el escenario Se envia un POST /api/users/login/ con credenciales incorrectas.	

Tipo de Evento Externo, iniciado por el actor.	
Garantia de exito (FALLO) No se entrega ningun token. El sistema no distingue entre "usuario inexistente" y "contrasena incorrecta" en el mensaje, para no filtrar que cuentas existen.	
Paso a paso	
Paso 1	El actor envia username y password incorrectos.
Paso 2	SimpleJWT intenta autenticar y falla.
Paso 3	El sistema responde 401 con un mensaje generico de credenciales invalidas.
Paso 4	El frontend muestra el error sin indicar cual de los dos datos estuvo mal.
Post Condiciones No se genera sesion ni tokens. El estado de la base de datos no cambia.	
Cuestiones pendientes No hay bloqueo por intentos fallidos repetidos (rate limiting); queda expuesto a fuerza bruta si no se agrega a futuro.	

Nombre del caso de uso Recuperacion de contraseña	
ID # #4.1	
Actores Usuario registrado (que olvido su contraseña)	
Precondiciones El usuario tiene una cuenta activa (is_active=True) con un email valido.	
Suposiciones El usuario tiene acceso a su casilla de correo.	
Evento que desencadena el escenario El usuario pide un reseteo (POST /api/users/password-reset/) y luego confirma con el link recibido (POST /api/users/password-reset/confirm/).	
Tipo de Evento Externo, en dos pasos (solicitud + confirmacion), con token de un solo uso.	
Garantia de exito (EXITO) La contraseña del usuario se actualiza (User.set_password + save), quedando hasheada correctamente.	
Paso a paso	
Paso 1	El usuario ingresa su email en el formulario de recuperacion.
Paso 2	El sistema busca un User activo con ese email; si existe, genera un token de reseteo y envia el email.
Paso 3	El sistema responde siempre con el mismo mensaje generico, exista o no el email, para no revelar que cuentas estan registradas.
Paso 4	El usuario hace clic en el link del email y define una nueva contraseña (password + password_confirm).
Paso 5	El sistema valida que ambas contraseñas coincidan.
Paso 6	El sistema verifica el token con check_reset_token(user, token).
Paso 7	El sistema aplica user.set_password(nueva_password) y guarda.
Paso 8	El sistema responde 200 con "Contraseña actualizada correctamente."
Paso 9	El usuario puede iniciar sesion con la nueva contraseña (ver #3.1).
Post Condiciones La contraseña del usuario queda actualizada y hasheada. El token de reseteo queda invalidado para reutilizacion.	
Cuestiones pendientes Ninguna.	

Nombre del caso de uso Confirmacion de nueva contraseña con datos que no coinciden	
ID # #4.2	

Actores Usuario registrado	
Precondiciones El usuario ya recibio el link de reseteo valido.	
Suposiciones El usuario tipea mal la confirmacion de la nueva contrasena.	
Evento que desencadena el escenario El usuario envia password y password_confirm con valores distintos en POST /api/users/password-reset/confirm/.	
Tipo de Evento Externo, iniciado por el actor.	
Garantia de exito (FALLO) La contrasena actual del usuario NO se modifica.	
Paso a paso	
Paso 1	El usuario completa el formulario de nueva contrasena con dos valores distintos.
Paso 2	El serializer (ResetPasswordSerializer.validate) detecta que password no es igual a password_confirm.
Paso 3	El sistema responde 400 con el error en password_confirm: "Las contrasenas no coinciden."
Paso 4	El usuario corrige el formulario y reintenta con el mismo link (todavia valido).
Post Condiciones La contrasena original del usuario sigue vigente. El token de reseteo sigue siendo valido para un nuevo intento.	
Cuestiones pendientes Ninguna.	

Nombre del caso de uso Edicion del perfil propio	
ID # #5.1	
Actores Usuario registrado (autenticado)	
Precondiciones El usuario tiene sesion iniciada (token JWT valido).	
Suposiciones El usuario quiere actualizar su bio y/o avatar.	
Evento que desencadena el escenario El usuario envia PATCH/PUT a /api/users/profile/ con los nuevos datos.	
Tipo de Evento Externo, iniciado por el actor autenticado.	
Garantia de exito (EXITO) El Profile asociado al usuario (relacion 1 a 1, creado automaticamente con get_or_create si no existia) queda actualizado con los nuevos valores.	
Paso a paso	
Paso 1	El usuario autenticado envia los campos bio y/o avatar a actualizar.
Paso 2	El sistema obtiene (o crea si no existia) el Profile del usuario con get_or_create.
Paso 3	El serializer valida que la bio no supere los 500 caracteres.
Paso 4	El sistema guarda los cambios en el Profile.
Paso 5	El sistema responde 200 con los datos actualizados (username, email, bio, avatar).
Post Condiciones El Profile queda actualizado y visible tanto en el perfil propio como en el perfil publico (ver #7.1).	
Cuestiones pendientes Ninguna.	

Nombre del caso de uso Edicion de perfil con bio demasiado larga	
ID # #5.2	
Actores Usuario registrado	
Precondiciones El usuario tiene sesion iniciada.	
Suposiciones El usuario pega un texto muy largo en el campo bio.	
Evento que desencadena el escenario El usuario envia una bio de mas de 500 caracteres.	

Tipo de Evento Externo, iniciado por el actor.	
Garantia de exito (FALLO) El Profile no se modifica; se conserva la bio anterior.	
Paso a paso	
Paso 1	El usuario envia una bio con mas de 500 caracteres.
Paso 2	El serializer (ProfileSerializer.validate_bio) detecta que supera el limite.
Paso 3	El sistema responde 400 con "La bio no puede superar los 500 caracteres."
Paso 4	El usuario acorta el texto y reintenta.
Post Condiciones El Profile conserva los valores previos a la edicion fallida.	
Cuestiones pendientes Ninguna.	

Nombre del caso de uso Agregar un juego a mi coleccion	
ID # #6.1	
Actores Usuario registrado (autenticado)	
Precondiciones El juego ya existe en el catalogo (Juego), cargado previamente por un Admin (ver #9.1). El usuario todavia no tiene ese juego en su coleccion personal.	
Suposiciones El usuario encontro el juego mediante la exploracion publica (ver #7.1).	
Evento que desencadena el escenario El usuario envia POST /api/juegos/mis-juegos/ con juego_id, estado, puntaje y/o resenia.	
Tipo de Evento Externo, iniciado por el actor autenticado.	
Garantia de exito (EXITO) Se crea un registro UserJuego que vincula a ese usuario con ese juego, con su estado, puntaje y resena personales.	
Paso a paso	
Paso 1	El usuario autenticado elige un juego del catalogo y define su estado (jugando, completado, pendiente o abandonado).
Paso 2	El sistema valida el serializer (UserJuegoSerializer).
Paso 3	El sistema verifica que no exista ya un UserJuego para ese mismo usuario y juego.
Paso 4	El sistema guarda el UserJuego asociando automaticamente al usuario autenticado (perform_create fuerza user=request.user).
Paso 5	El sistema responde 201 con los datos del UserJuego creado, incluyendo el detalle completo del juego.
Post Condiciones El juego pasa a formar parte de la coleccion personal del usuario y puede verse en sus estadisticas (ver #8.1).	
Cuestiones pendientes Ninguna.	

Nombre del caso de uso Agregar un juego que ya esta en la coleccion	
ID # #6.2	
Actores Usuario registrado	
Precondiciones El usuario ya tiene un UserJuego previo para ese mismo juego.	
Suposiciones El usuario no se da cuenta de que ya lo habia agregado antes.	

Evento que desencadena el escenario El usuario intenta agregar de nuevo el mismo juego a su coleccion.	
Tipo de Evento Externo, iniciado por el actor.	
Garantia de exito (FALLO) No se crea un UserJuego duplicado; se respeta la restricción unique_together (user, juego) definida en el modelo.	
Paso a paso	
Paso 1	El usuario envia POST /api/juegos/mis-juegos/ con un juego_id que ya tiene asociado.
Paso 2	El serializer (UserJuegoSerializer.validate) verifica si ya existe un UserJuego para ese usuario y ese juego.
Paso 3	El sistema responde 400 con "Ya estas en este juego."
Paso 4	El usuario puede en cambio editar el registro existente (PATCH) para actualizar estado o puntaje.
Post Condiciones La coleccion del usuario no cambia; sigue teniendo un unico registro para ese juego.	
Cuestiones pendientes Ninguna.	

Nombre del caso de uso Puntuar y reseñar un juego de mi coleccion	
ID # #6.3	
Actores Usuario registrado	
Precondiciones El usuario ya tiene el juego en su coleccion (UserJuego existente, ver #6.1).	
Suposiciones El usuario termino de jugarlo o quiere dejar su opinion.	
Evento que desencadena el escenario El usuario envia PATCH a /api/juegos/mis-juegos/{id}/ con puntaje (1-10) y/o resenia.	
Tipo de Evento Externo, iniciado por el actor.	
Garantia de exito (EXITO) El UserJuego queda actualizado con el nuevo puntaje y/o resena.	
Paso a paso	
Paso 1	El usuario edita su UserJuego existente con un puntaje entre 1 y 10 y un texto de resena.
Paso 2	El serializer valida el rango del puntaje (validate_puntaje).
Paso 3	El sistema guarda los cambios.
Paso 4	El sistema responde 200 con los datos actualizados.

Post Condiciones El puntaje y la resena quedan disponibles para el calculo de estadisticas (promedio de puntaje, ver #8.1).
Cuestiones pendientes Ninguna.

Nombre del caso de uso Puntuar un juego fuera del rango permitido	
ID # #6.4	
Actores Usuario registrado	
Precondiciones El usuario tiene un UserJuego existente.	
Suposiciones El usuario ingresa un valor fuera de la escala esperada (por ejemplo 15).	
Evento que desencadena el escenario El usuario envia un puntaje menor a 1 o mayor a 10.	
Tipo de Evento Externo, iniciado por el actor.	
Garantia de exito (FALLO) El puntaje no se guarda; se conserva el valor anterior (o null si nunca puntuo).	
Paso a paso	
Paso 1	El usuario envia puntaje=15 (o cualquier valor fuera de 1-10).
Paso 2	El serializer (UserJuegoSerializer.validate_puntaje) detecta que el valor esta fuera de rango.
Paso 3	El sistema responde 400 con "El puntaje debe estar entre 1 y 10."
Paso 4	El usuario corrige el valor y reintenta.
Post Condiciones El UserJuego conserva su puntaje previo, sin cambios.	
Cuestiones pendientes Ninguna.	

Nombre del caso de uso Explorar y filtrar el catalogo publico de juegos	
ID # #7.1	
Actores Visitante o Usuario registrado	
Precondiciones Existen juegos cargados en el catalogo (Juego).	
Suposiciones El actor quiere descubrir juegos nuevos, con o sin filtros aplicados.	
Evento que desencadena el escenario El actor consulta GET /api/juegos/publico/ con parametros opcionales de filtro (genero, plataforma) y orden.	
Tipo de Evento Externo, iniciado por el actor; accesible sin autenticacion (AllowAnonymous).	
Garantia de exito (EXITO) Se devuelve el listado de juegos del catalogo, filtrado y ordenado segun lo pedido, sin exponer datos personales de ningun usuario.	
Paso a paso	
Paso 1	El actor abre la seccion de exploracion (con o sin sesion iniciada).
Paso 2	El sistema aplica el JuegoFilter (por genero y/o plataforma) mediante django-filter.
Paso 3	El sistema aplica el ordering solicitado (por created_at o nombre).
Paso 4	El sistema devuelve la lista paginada de juegos con sus datos publicos (nombre, genero, plataforma, imagen, descripcion, ano).
Post Condiciones El actor puede elegir un juego y, si esta autenticado, agregarlo a su coleccion (ver #6.1).	
Cuestiones pendientes El README menciona ordenar por puntaje y ver la coleccion de otros usuarios, pero el endpoint publico de juegos no expone el puntaje personal de cada usuario (eso vive en UserJuego, que es privado por diseno).	

Nombre del caso de uso Ver el detalle de un juego	
ID # #7.2	
Actores Visitante o Usuario registrado	
Precondiciones El juego solicitado existe en el catalogo.	
Suposiciones El actor hizo clic en un juego desde el listado de exploracion.	
Evento que desencadena el escenario El actor consulta GET /api/juegos/{juego_id}/detalle/.	

Tipo de Evento Externo, accesible sin autenticacion (AllowAny).	
Garantia de exito (EXITO) Se devuelve el juego junto con el puntaje promedio de todos los usuarios que lo puntuaron, el total de resenas, y si el actor esta autenticado, su propio progreso personal (estado, puntaje y resena).	
Paso a paso	
Paso 1	El actor abre el detalle de un juego desde la exploracion.
Paso 2	El sistema busca el Juego por su id; si existe, continua.
Paso 3	El sistema calcula el promedio de puntaje y el total de resenas entre todos los UserJuego asociados a ese juego.
Paso 4	El sistema serializa los datos generales del juego.
Paso 5	Si el actor esta autenticado, el sistema busca si tiene un UserJuego propio para ese juego y lo incluye en la respuesta.
Paso 6	El sistema responde 200 con juego, puntaje_promedio, total_resenas y mi_user_juego (null si no esta autenticado o no lo tiene en su coleccion).
Post Condiciones No se modifica ningun dato; es una consulta de solo lectura.	
Cuestiones pendientes Ninguna.	

Nombre del caso de uso Ver el detalle de un juego que no existe	
ID # #7.3	
Actores Visitante o Usuario registrado	
Precondiciones El id de juego solicitado no existe en el catalogo (fue eliminado o el id es incorrecto).	
Suposiciones El actor accede a una URL vieja o mal escrita.	
Evento que desencadena el escenario El actor consulta GET /api/juegos/{juego_id}/detalle/ con un id inexistente.	
Tipo de Evento Externo, iniciado por el actor.	
Garantia de exito (FALLO) No se devuelve ningun dato del juego; el sistema responde con un error claro en vez de romperse.	
Paso a paso	
Paso 1	El actor accede a la URL de detalle con un id que no existe.
Paso 2	El sistema intenta buscar el Juego por ese id y detecta que no existe.
Paso 3	El sistema responde 404 con el codigo "not_found" y el mensaje "Juego no encontrado".

Post Condiciones No se modifica ningun dato.
Cuestiones pendientes Ninguna.

Nombre del caso de uso Ver quien mas tiene un juego en su coleccion
ID # #7.4
Actores Visitante o Usuario registrado
Precondiciones Al menos un usuario tiene ese juego en su coleccion.
Suposiciones El actor quiere ver quien mas juega ese titulo, o buscar si un usuario puntual lo tiene.
Evento que desencadena el escenario El actor consulta GET /api/juegos/{juego_id}/jugadores/ con un parametro opcional q para buscar por nombre de usuario.
Tipo de Evento Externo, accesible sin autenticacion (AllowAny).
Garantia de exito (EXITO) Se devuelve una lista paginada (10 por pagina) de usuarios que tienen ese juego, mostrando solo username, estado y puntaje, nunca el email ni otros datos sensibles.

Paso a paso	
Paso 1	El actor abre la seccion de jugadores dentro del detalle de un juego.
Paso 2	El sistema filtra los UserJuego de ese juego_id.
Paso 3	Si se envio el parametro q, el sistema filtra ademas por username que contenga ese texto.
Paso 4	El sistema pagina el resultado (10 por pagina).
Paso 5	El sistema responde 200 con la lista de jugadores (username, estado, puntaje), el total y la info de paginacion.
Post Condiciones No se modifica ningun dato.	
Cuestiones pendientes Ninguna.	

Nombre del caso de uso Buscar juegos por una plataforma abreviada
ID # #7.5
Actores Visitante o Usuario registrado

Precondiciones Existen juegos en el catalogo cargados con plataformas como "PlayStation 5".	
Suposiciones El actor escribe una forma corta y natural de buscar, en vez del nombre completo.	
Evento que desencadena el escenario El actor consulta GET /api/juegos/publico/?plataforma=PS5.	
Tipo de Evento Externo, iniciado por el actor.	
Garantia de exito (EXITO) El filtro reconoce la abreviatura y devuelve los juegos correspondientes, aunque el texto no coincida literalmente con el valor guardado en la base.	
Paso a paso	
Paso 1	El actor busca juegos escribiendo "PS5" en el filtro de plataforma.
Paso 2	El sistema detecta con una expresion regular que el texto matchea el patron "ps" mas un numero opcional.
Paso 3	El sistema traduce la busqueda a "PlayStation 5" y aplica un filtro de coincidencia parcial sobre el campo plataforma.
Paso 4	El sistema devuelve todos los juegos cuya plataforma contenga "PlayStation 5".
Post Condiciones No se modifica ningun dato.	
Cuestiones pendientes Si el actor escribe una plataforma que no sea PlayStation (por ejemplo Xbox), la busqueda usa el texto tal cual, sin ninguna traduccion especial.	

Nombre del caso de uso Ver estadísticas de mi coleccion	
ID # #8.1	
Actores Usuario registrado (autenticado)	
Precondiciones El usuario tiene al menos un UserJuego en su coleccion (si no tiene ninguno, las metricas devuelven 0).	
Suposiciones El usuario quiere ver un resumen de su actividad.	
Evento que desencadena el escenario El usuario consulta GET /api/juegos/mis-juegos/estadisticas/.	
Tipo de Evento Externo, iniciado por el actor autenticado.	
Garantia de exito (EXITO) Se calculan y devuelven las metricas en tiempo real: cantidad de juegos completados, promedio de puntaje y generos mas jugados.	
Paso a paso	
Paso 1	El usuario autenticado accede a la seccion de estadisticas.
Paso 2	El sistema filtra los UserJuego de ese usuario (el queryset ya esta limitado por user=request.user).
Paso 3	El sistema cuenta cuantos tienen estado='completado'.
Paso 4	El sistema calcula el promedio de puntaje con Avg('puntaje'), redondeado a 1 decimal.
Paso 5	El sistema agrupa por genero del juego y cuenta ocurrencias, ordenado de mayor a menor.
Paso 6	El sistema responde 200 con juegos_completados, promedio_puntaje y generos_mas_jugados.
Post Condiciones No se modifica ningun dato; es una consulta de solo lectura.	
Cuestiones pendientes Todo se calcula al momento de la consulta (sin cache ni tareas en segundo plano), tal como aclara el README ("sin usar celery").	

Nombre del caso de uso Admin agrega un juego al catalogo	
ID # #9.1	
Actores Admin (User con is_staff=True)	
Precondiciones El actor tiene sesion iniciada y su cuenta tiene is_staff=True.	
Suposiciones El juego que se quiere agregar existe en la base de datos de RAWG y todavia no esta en el catalogo local.	
Evento que desencadena el escenario El Admin busca el juego en RAWG (GET /api/juegos/rawg/buscar/) y luego confirma la creacion (POST /api/juegos/, solo permitido si IsAdminUser).	
Tipo de Evento Externo, iniciado por el actor; protegido por permiso de administrador.	
Garantia de exito (EXITO) Se crea un nuevo registro Juego en el catalogo local, disponible para todos los usuarios en la exploracion publica.	
Paso a paso	
Paso 1	El Admin busca un titulo en RawgBuscarView, que consulta la API externa de RAWG.
Paso 2	El Admin selecciona el resultado deseado y el sistema arma los campos (nombre, genero, plataforma, imagen, descripcion, fecha_lanzamiento) a partir de la respuesta de RAWG.
Paso 3	El sistema valida que no exista ya un Juego con el mismo nombre (sin distinguir mayusculas) ni con el mismo rawg_id.
Paso 4	El sistema crea el registro Juego en el catalogo.
Paso 5	El sistema responde 201 con los datos del juego creado.
Post Condiciones El juego queda disponible en el catalogo publico (ver #7.1) para que cualquier usuario lo agregue a su coleccion.	
Cuestiones pendientes Ninguna.	

Nombre del caso de uso Usuario sin permisos intenta crear un juego	
ID # #9.2	
Actores Usuario registrado (sin permisos de Admin)	
Precondiciones El actor tiene sesion iniciada pero is_staff=False.	
Suposiciones El usuario intenta acceder directamente al endpoint (por API o inspeccionando el frontend), sin tener el rol de Admin.	

Evento que desencadena el escenario El usuario envia POST /api/juegos/ para crear un juego en el catalogo.	
Tipo de Evento Externo, iniciado por el actor; bloqueado por control de permisos.	
Garantia de exito (FALLO) No se crea ningun Juego. El catalogo permanece sin cambios.	
Paso a paso	
Paso 1	El usuario autenticado (no admin) envia una solicitud para crear un juego.
Paso 2	JuegoViewSet.get_permissions() detecta que la accion es 'create' y exige IsAdminUser.
Paso 3	DRF verifica request.user.is_staff y encuentra que es False.
Paso 4	El sistema responde 403 (Forbidden) antes de ejecutar cualquier logica de creacion.
Post Condiciones El catalogo de juegos no se modifica.	
Cuestiones pendientes La misma regla protege editar (update) y eliminar (destroy) juegos del catalogo.	

Nombre del caso de uso Admin elimina un usuario	
ID # #10.1	
Actores Admin	
Precondiciones El actor tiene is_staff=True. Existe otro usuario (distinto al Admin) a eliminar.	
Suposiciones El Admin decidio dar de baja una cuenta (por ejemplo, por mal uso de la plataforma).	
Evento que desencadena el escenario El Admin envia DELETE /api/users/admin/users/{id}/delete/.	
Tipo de Evento Externo, iniciado por el actor; protegido por permiso de administrador.	
Garantia de exito (EXITO) El User indicado se elimina de la base de datos, junto con sus datos relacionados en cascada (Profile y UserJuego, por on_delete=CASCADE).	
Paso a paso	
Paso 1	El Admin abre el listado de usuarios (AdminUserListView) y elige uno para eliminar.
Paso 2	El sistema verifica que request.user.is_staff sea True.
Paso 3	El sistema verifica que el usuario a eliminar no sea el mismo Admin que hace la solicitud.
Paso 4	El sistema elimina el User, lo que dispara en cascada el borrado de su Profile y sus UserJuego.
Paso 5	El sistema responde 200 con "Usuario '{username}' eliminado correctamente."
Post Condiciones El usuario eliminado ya no puede iniciar sesion ni aparece en ningun listado. Su coleccion de juegos desaparece junto con el.	
Cuestiones pendientes El borrado es definitivo (no hay soft-delete ni papelera de reciclaje); una eliminacion accidental no se puede deshacer.	

Nombre del caso de uso Admin intenta eliminarse a si mismo	
ID # #10.2	
Actores Admin	
Precondiciones El actor tiene is_staff=True.	
Suposiciones El Admin se equivoca de fila en el listado, o prueba el endpoint sobre su propio ID.	
Evento que desencadena el escenario El Admin envia DELETE /api/users/admin/users/{su_propio_id}/delete/.	

Tipo de Evento Externo, iniciado por el actor; bloqueado por regla de negocio explicita.	
Garantia de exito (FALLO) La cuenta del Admin no se elimina, evitando que el sistema se quede sin administradores por accidente.	
Paso a paso	
Paso 1	El Admin envia una solicitud de eliminacion usando su propio ID de usuario.
Paso 2	El sistema confirma que request.user.is_staff es True (pasa el primer control).
Paso 3	El sistema compara el usuario objetivo contra request.user y detecta que coinciden.
Paso 4	El sistema responde 400 con el codigo "self_delete" y el mensaje "No podes eliminar tu propio usuario."
Post Condiciones La cuenta del Admin sigue activa y sin cambios.	
Cuestiones pendientes Si un Admin necesita eliminar su propia cuenta, necesitaria que otro Admin lo haga por el (no hay atajo de auto-baja).	

Nombre del caso de uso Admin busca usuarios por el nombre de un juego	
ID # #10.3	
Actores Admin	
Precondiciones El actor tiene is_staff=True. Existen usuarios con juegos en su coleccion.	
Suposiciones El Admin quiere encontrar rapidamente a los usuarios que tienen un titulo puntual (por ejemplo, para soporte o moderacion).	
Evento que desencadena el escenario El Admin consulta GET /api/users/admin/users/?search=.	
Tipo de Evento Externo, iniciado por el actor; protegido por permiso de administrador.	
Garantia de exito (EXITO) Se devuelve la lista de usuarios cuyo username, email o alguno de los juegos de su coleccion coincide con el texto buscado.	
Paso a paso	
Paso 1	El Admin escribe el nombre de un juego (o parte de el) en el buscador del listado de usuarios.
Paso 2	El sistema confirma que request.user.is_staff es True.
Paso 3	El sistema aplica la busqueda sobre username, email y el nombre de los juegos de la coleccion de cada usuario.
Paso 4	El sistema devuelve, sin duplicados, los usuarios que coinciden, junto con la lista de nombres de juegos de cada uno.

Post Condiciones No se modifica ningun dato.
Cuestiones pendientes Ninguna.

Nombre del caso de uso Usuario sin permisos intenta ver el listado de usuarios	
ID # #10.4	
Actores Usuario registrado (sin permisos de Admin)	
Precondiciones El actor tiene sesion iniciada pero is_staff=False.	
Suposiciones El usuario intenta acceder directamente al endpoint de administracion.	
Evento que desencadena el escenario El usuario envia GET /api/users/admin/users/.	
Tipo de Evento Externo, iniciado por el actor; bloqueado por control de permisos.	
Garantia de exito (FALLO) No se devuelve ningun dato de otros usuarios; se protege la privacidad del resto de la base de usuarios.	
Paso a paso	
Paso 1	El usuario autenticado (no admin) intenta acceder al listado de usuarios.
Paso 2	El sistema verifica request.user.is_staff y encuentra que es False.
Paso 3	El sistema responde 403 con el codigo "permission_denied" y el mensaje "No tenes permisos de administrador."
Post Condiciones No se modifica ningun dato; ningun dato de otros usuarios se expone.	
Cuestiones pendientes Ninguna.	